

Título: Documentación de desarrollo de software

Materia: Orientación a Objetos 2

Departamento: Desarrollo Productivo y Tecnológico

Docente: Gustavo Hernán Siciliano

Asistente: Oscar Ruina

Integrantes:

Arturo Eduartes

Gastón Romero

Ezequiel Gonzalez

Tomas Diaz De Maio

DNI:

18.511.610

45.873.095

42.397.100

43.895.372

Fecha de entrega: 19/06/2024

Link de los videos de defensa del TP **4**
Guía de Reproducción del Proyecto **5**
Clonación del proyecto 5
Creación de la Base de datos 5
Correr el proyecto 5
Acceder al Proyecto 5
Vista Home 6
Vista Producto (Como Admin) 6
Vista Producto (Como Cliente) 6
Vista Ventas (Como Admin) 7
Vista Historial (Como Cliente) 7
Vista Compras (Exclusiva de Admin) 7
Vista Pedidos de Compra (Exclusiva de Admin) 7
Vista Logout 7
Diagramas **8**
Diagrama de entidad relación **8**
Diagrama de clases **9**
Aclaraciones previas al desarrollo de los casos de usos 10
1 - Iniciar sesión (Cliente/Admin) 11
Curso normal 12
Curso alternativo 12
2 - Realizar logout de la aplicación (Cliente/Admin) 13
Curso normal 14
Curso alternativo 14
3 - Realizar redirección a la página principal (Cliente/Admin) 14
Curso normal 16
Curso alternativo 16
4 - Realizar alta de producto (Admin) 17
Curso normal 18
Curso alternativo 19
5 - Realizar modificación de producto (Admin) 20
Curso normal 21
Curso alternativo 21
6 - Realizar baja de producto (Admin) 22
Curso normal 23
Curso alternativo 23
7 - Realizar compra de producto (Cliente) 24
Curso normal 25
Curso alternativo 25
8 - Revisar compra de producto (Cliente) 26
Curso normal 27
Curso alternativo 27
9 - Realizar reposición de stock (Admin) 28

Curso normal 29
Curso alternativo 30
10 - Realizar historial de ventas (Cliente/Admin) 31
Curso normal 32
Curso alternativo 32

Link de los videos de defensa del TP

https://drive.google.com/drive/folders/1FGDLHxGmQeV2qPMFXKnJ33ZCxi8uNTEu?usp=drive_link

Guía de Reproducción del Proyecto

Clonación del proyecto

Se requiere clonar el proyecto con la última actualización en la rama main

https://github.com/EzequielDeiana/grupo-2-OO2-2024

Se deben crear variables de entorno para los siguientes valores:

${DB_URL}

${DB_USERNAME}

${DB_PASSWORD}

Por ejemplo:

Url: jdbc:mysql://localhost:Puerto de la BD/tp_grupo2

Username: root

Password: root1234

Creación de la Base de datos

Dentro del repositorio se encuentra el archivo Script_Create_Schema_and_Data.sql

Al ejecutarlo se creará una base de datos con el nombre “tp_grupo2” con las tablas, columnas y datos ya incluidos.

Correr el proyecto

Se utiliza la versión de JAVA 17 y MySQL8.

El proyecto va a correr en el puerto 8080 nativo de Spring.

Ejemplo: “http://localhost:8080/login”. Se requiere la disponibilidad del mismo

Acceder al Proyecto

Al acceder a cualquier endpoint de la página. Se recomienda ingresar a “http://localhost:8080/index”, desde donde se va a redirigir a “http://localhost:8080/login” donde va a requerir ingresar los datos de un usuario.

Para acceder como admin están los siguientes posibles perfiles:

Username: Ezequiel - Contraseña: 1234

Username: Gaston - Contraseña: 1234

Username: Arturo - Contraseña: 1234

Username: Tomas - Contraseña: 1234

Para acceder como cliente están los siguientes posibles perfiles:

Username: Rodrigo - Contraseña: 1234

Username: Pedro - Contraseña: 1234

Username: Pepito - Contraseña: 1234

Username: Jorge - Contraseña: 1234

Según el rol del usuario con el que se haya ingresado se renderizan diferentes elementos de la vista del HTML a la vez que se limitarán los accesos a distintos endpoints.

Vista Home

"http://localhost:8080/index"

Al acceder al index del proyecto se tendrá un mensaje de bienvenida.

En la vista de navegación aparecen opciones que te permitirá redireccionar a distintas vistas. Estan opciones van a variar según si se está logueado como Cliente o como Admin

Vista Producto (Como Admin)

"http://localhost:8080/producto/index"

Al ingresar a la vista de productos con el rol admin se va a poder ver la lista de productos cargados en la base de datos sin importar si están disponibles o no.

Se puede utilizar el botón [Modificar] para acceder al producto en específico y modificar sus datos.

Debajo de la lista de productos está el botón [Crear nuevo Producto] que dará la posibilidad de crear un nuevo producto desde 0.

Por último, debajo del botón están dos listas con el historial de ganancias totales de cada producto que tengamos en la base de datos.

Vista Producto (Como Cliente)

“http://localhost:8080/producto/index”

Al ingresar a la vista de productos con el rol cliente se va a poder ver la lista de productos cargados en la base de datos cuales tengan disponibilidad de stock y que estén disponibles. El sistema no permite ver al cliente los productos que estén dados de baja de forma lógica.

En la tabla de productos se habilita la tabla de realizar compra en la cual se podrá especificar una cantidad a comprar, al hacer click en el botón [Comprar] se hará la baja de la cantidad señalada en el stock y se sumará el precio al total vendido.

Vista Ventas (Como Admin)

“http://localhost:8080/venta/index”

En esta vista se puede ver el historial de compras de cada usuario con su username, producto, cantidad y fecha de la realización de la compra.

Vista Historial (Como Cliente)

“http://localhost:8080/venta/historial”

En esta vista se puede ver el historial de compras realizadas por el cliente que está logueado.

Vista Compras (Exclusiva de Admin)

“http://localhost:8080/compra/index”

En esta vista se puede ver el historial de compras que realizó el negocio para reabastecer el stock. Se incluye el id que lo relaciona con el pedido de compra, la fecha de entrega, la fecha en que se lanzó la alerta de la necesidad de reabastecimiento, el proveedor, la cantidad comprada y el producto.

Vista Pedidos de Compra (Exclusiva de Admin)

“http://localhost:8080/pedidocompra/index”

En esta vista se pueden ver las alertas realizadas por el sistema ante la baja cantidad de stock con la opción de realizar la compra. Al hacer click se podrá acceder a la vista para seleccionar la cantidad a comprar y seleccionar el proveedor.

Vista Logout

Se accedera a esta vista al dar click a la opción de logout en cualquiera de las otras vistas. Hace una redirección exitosa al Login con un mensaje de éxito al cerrar sesión.

Diagramas

Diagrama de entidad relación

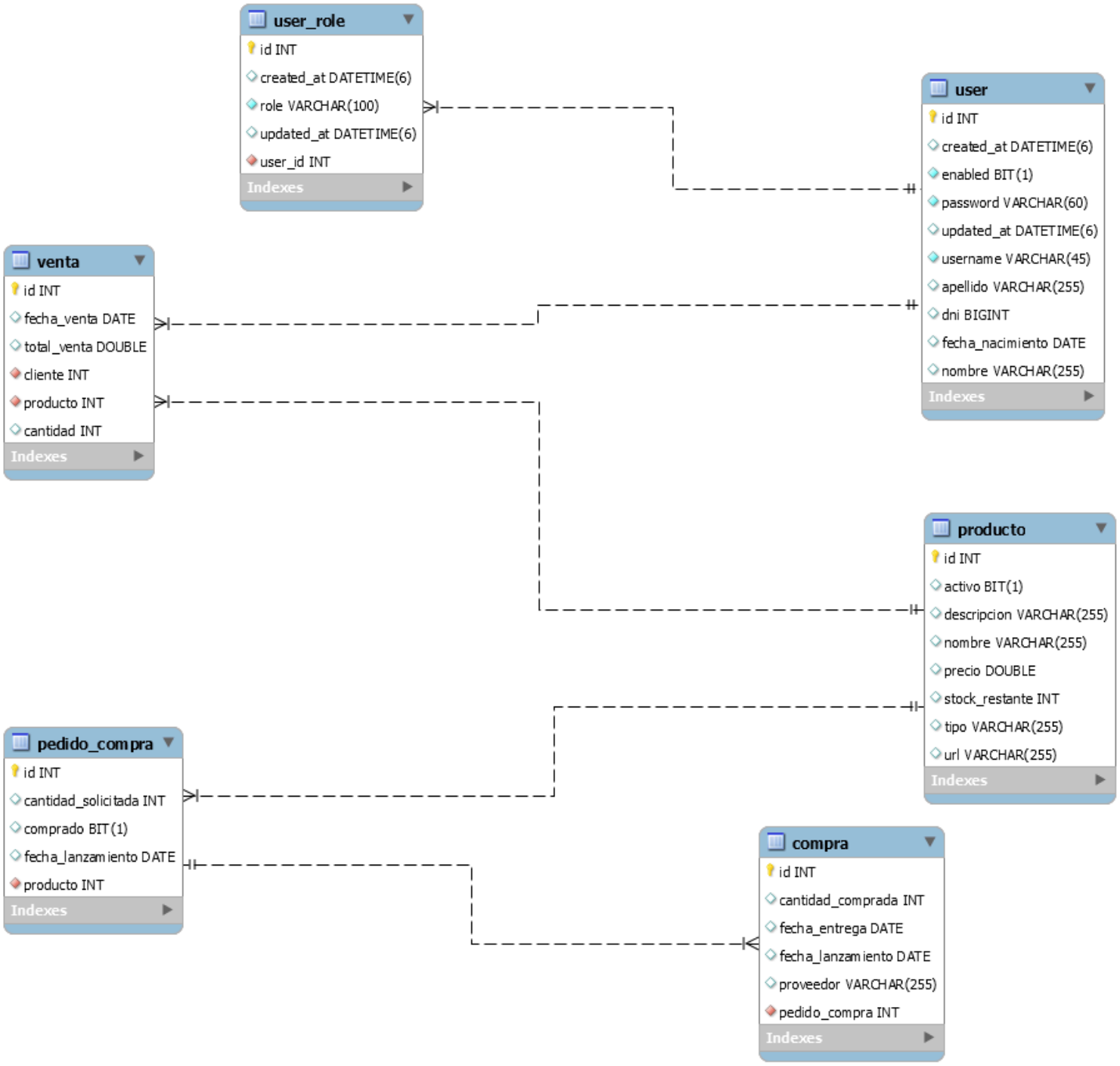
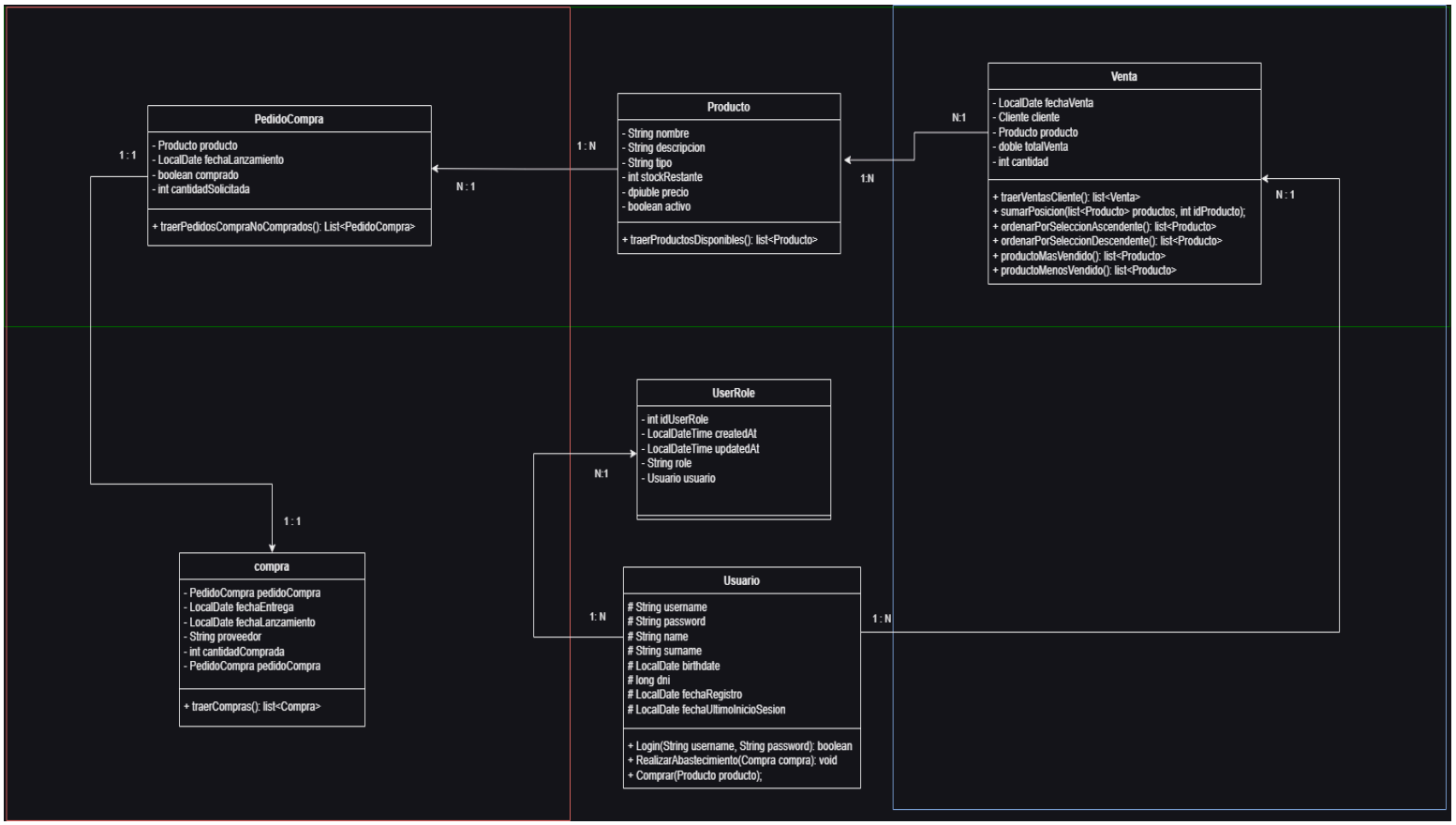


Diagrama de clases



Aclaraciones previas al desarrollo de los casos de usos

Al principio hemos desarrollado en conjunto las bases como el login, logout, Security Configuration, Custom Error Controller, etc. Nos hemos reunido mediante una plataforma llamada (discord) en donde una persona se encargaba de "*codear*" mientras que el resto realizaba proposiciones de resolución de errores o conflictos en el código, incluyendo planteamientos de modificación de entidades, atributos, algoritmos, etc.

Posteriormente a la construcción de las bases, el proyecto ha sido desarrollado por capas en lugar de por componentes, (primero el Controller, luego el Service, luego el Repository, y finalmente los Templates). En dónde una persona no realizó de manera integral todas las características del proyecto, es decir, que una persona solo realizó el commit de todos los Controller, otra persona realizó el commit de todos los Service, otra persona realizó el commit de todos los Repository, y finalmente otra persona realizó el commit de todos los Templates (html).

Posterior a la implementación de la primera versión de todas las clases, cada uno ha modificado, corregido, actualizado de manera diversa cada una de estas clases, en donde la persona que desarrolló los Controller corrigió los Service, la persona que corrigió los Service actualizó los Template, la persona que desarrolló los Repository actualizó los Service, y la persona que desarrolló los Templates se encargó de corregir los Controllers, y así sucesivamente variando de componentes.

El 90% del proyecto ha sido realizado en llamada grupal, en dónde hemos participado directamente en la construcción del proyecto, mediante recomendaciones, acotaciones, discusiones, entre otros.

Por lo que cabe recalcar, que una baja cantidad de commits han sido realizados de manera individual sin apoyo o recomendaciones del resto de integrantes del proyecto.

El integrante Arturo Eduartes, tiene 2 cuentas asociadas al repositorio público, ArturoEduartes y ArturoEduartesUNLa, en ambas cuentas hay commits realizados y líneas de código insertadas y removidas. Esto se debe a que ocurrieron problemas vinculados a la autenticación de dichas cuentas durante el transcurso del proyecto.

1 - Iniciar sesión (Cliente/Admin)

Commit	Gaston Romero (Esqueleto)	
Revisado/Modificado	Ezequiel Gonzalez (fixes) Arturo Eduartes (fixes)	
Caso de uso	Iniciar sesión	Identificador: CU 1
Actores	Usuario	
Tipo	Primario	
Referencias	Usuario y Contraseña	
Precondición	El usuario y contraseña ingresados deben ser existentes en la base de datos y ser válidos.	
Postcondición	El Usuario inicia sesión en la página ingresando sus credenciales.	
Descripción	El Usuario (ya sea Admin o Cliente) intentará ingresar al sistema mediante el login.	
Resumen	Se revisó en conjunto el TFI publicado para ver cómo funcionaba y se podía implementar el sistema de login, encriptación y validez del usuario. Mientras se estaba en reunión la creación del caso de uso se hizo con la participación activa del grupo en cuanto a investigación y recomendaciones. Se describe al ejecutor como User porque tanto Clientes como Admins pueden usar el mismo sistema de Login. Controller:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo2/src/main/java/com/unla/oo2/grupo2/controller/UserController.java Service:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo2/src/main/java/com/unla/oo2/grupo2/service/UserService.java Vista:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo2/src/main/resources/templates/user/login.html	

Curso normal

Nro	Ejecutor	Paso o actividad
1	Usuario	Selecciona la casilla usuario
2	Usuario	Ingresa su usuario
3	Usuario	Selecciona la casilla contraseña
4	Usuario	Ingresa su contraseña
5	Usuario	Selecciona el botón de Login
6	Usuario	Logra iniciar sesión

Curso alternativo

Nro	Descripción de acciones alternas
6	El User recibe el mensaje: “Invalid username or password”. Referencia: Paso 6
7	El Usuario se comunica con el desarrollador
8	El Usuario inicia sesión con las credenciales válidas

2 - Realizar logout de la aplicación (Cliente/Admin)

Commit	Gaston Romero (Esqueleto)	
Revisado/Modificado	Ezequiel Gonzalez (fixes) Arturo Eduartes (fixes)	
Caso de uso	Cerrar sesión	Identificador: CU 2
Actores	Usuario	
Tipo	Primario	
Referencias	Login	
Precondición	El usuario tiene que estar logueado previamente en la página(CU 1)	
Postcondición	El usuario cierra la sesión y vuelve a la vista de Login	
Descripción	El Usuario (ya sea Admin o Cliente) intentará salir de la sesión.	
Resumen	Se revisó en conjunto el TFI publicado para ver cómo funcionaba y se podía implementar el sistema de logout y validez del usuario. Mientras se estaba en reunión la creación del caso de uso se hizo con la participación activa del grupo en cuanto a investigación y recomendaciones. Se describe al ejecutor como User porque tanto Clientes como Admins pueden usar el mismo sistema de Logout. Controller:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo2/src/main/java/com/unla/oo2/grupo2/controller/UserController.java Service:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo2/src/main/java/com/unla/oo2/grupo2/service/UserService.java Vista:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/template	

	s/user/logout.html

Curso normal

Nro	Ejecutor	Paso o actividad
1	Usuario	Hace click en el botón de logout en la navbar
2	Sistema	Redirecciona a la página de login
3	Sistema	Confirma el cierre de sesión con un mensaje de éxito

Curso alternativo

Nro	Descripción de acciones alternas
6	El usuario no logra cerrar sesión Referencia: 2
7	El usuario se comunica con el desarrollador

3 - Realizar redirección a la página principal (Cliente/Admin)

Commit	Gastón Romero (Esqueleto)	
Revisado/Modificado	Arturo Eduartes (updates) Tomás Díaz De Maio (updates) Ezequiel Gonzalez (updates)	
Caso de uso	Volver Página Principal	Identificador: CU 3
Actores	Usuario	
Tipo	Primario	
Referencias	Caso de uso relacionado: CU1	
Precondición	El cliente debe haber iniciado sesión y accedido a otra pestaña	
Postcondición	El cliente es redirigido a la página principal	
Descripción	El cliente desea ir a la página principal	
Resumen	Se realizó una navbar para navegar entre las distintas direcciones de la página web, estando entre una de las posibilidades el ir al /index pulsando el botón “Home” el cuál tiene vinculado un href que redirecciona a “http://localhost:8080/index”. Se describe al ejecutor como User porque tanto Clientes como Admins pueden usar el mismo sistema de redirección con el botón Home <li class="nav-item">Home Controller:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/controller/HomeController.java Vista:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/templates/home/index.html	

Curso normal

Nro	Ejecutor	Paso o actividad
1	Cliente	Selecciona el botón Home
2	Cliente	Se lo redirige a la página principal

Curso alternativo

Nro	Descripción de acciones alternas
6	El cliente no es enviado a la página principal. Referencia: Paso 2
7	El cliente se comunica con el administrador

4 - Realizar alta de producto (Admin)

Commit	Ezequiel Gonzalez (Esqueleto y vistas)	
Revisado/Modificado	Arturo Eduartes (capa Service) Tomás De Maio (capa Controller)	
Caso de uso	Dar Alta Producto	Identificador: CU 4
Actores	Administrador	
Tipo	Primario	
Referencias	Iniciar sesión como administrador e ingresar a la pestaña Productos. Hará click en el botón “crear nuevo Producto” e ingresará al formulario de Creación de un producto. Caso de uso relacionado: CU1	
Precondición	Nombre, Descripción, Tipo, Stock Restante, Precio, Imagen y Estado del producto a agregar	
Postcondición	Se genera el producto en el sistema	
Descripción	Se desea agregar un producto al sistema	
Resumen	Se creó un método add(Producto producto) en serviceInterfaces, el cual tiene la acción de .save en service al extender desde el repository. En el controller se utiliza el método getMapping(/new) para cargar en la vista el modelo con el nuevo objeto de tipo Producto. En el formulario creado en (producto/new) se utilizó Thymeleaf para manejar el método post, crear el objeto y cargar los campos usando los inputs del formulario. Al enviar el formulario se activa el método postMapping(/create) para agregar el nuevo objeto de tipo Producto con los asignados a la base de datos Controller:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/controller/ProductoController.java	

	Service:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/service/ProductoService.java Vista:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/templates/producto/new.html

Curso normal

Nro	Ejecutor	Paso o actividad
1	Admin	Selecciona el botón Producto
2	Admin	Recorre la tabla hasta encontrar el botón Crear nuevo producto
3	Admin	Selecciona el botón Crear nuevo producto
4	Admin	Selecciona la casilla Nombre
5	Admin	Ingresa el nombre correspondiente
6	Admin	Selecciona la casilla Descripción
7	Admin	Ingresa la descripción correspondiente
8	Admin	Selecciona la casilla Tipo
9	Admin	Ingresa el tipo correspondiente
10	Admin	Selecciona la casilla Stock Restante
11	Admin	Ingresa el stock correspondiente
12	Admin	Selecciona la casilla Precio
13	Admin	Ingresa el precio correspondiente
14	Admin	Selecciona la casilla Imagen
15	Admin	Ingresa el URL correspondiente
16	Admin	Selecciona la casilla Activo
17	Admin	Selecciona si es True o False

18	Sistema	Se agrega el producto al sistema

Curso alternativo

Nro	Descripción de acciones alternas
19	El producto no se añadió correctamente, redireccionando al administrador a un error. Referencia: Paso 18
20	Comunicarse con soporte técnico.

5 - Realizar modificación de producto (Admin)

Commit	Ezequiel Gonzalez (Esqueleto y vista)	
Revisado/Modificado	Arturo Eduartes (capa Service) Tomás De Maio (capa Controller)	
Caso de uso	Modificar Producto	Identificador: CU 5
Actores	Administrador	
Tipo	Primario	
Referencias	Iniciar sesión como administrador, ingresar a la pestaña producto y apretar el botón “Modificar producto”. Caso de uso relacionado: CU1	
Precondición	Deben existir productos en el sistema	
Postcondición	Se actualiza la información del producto	
Descripción	Se desea modificar un producto existente	
Resumen	Se creó un método findById(int id) en serviceInterfaces, el cual tiene el método findById por defecto en el productRepository para obtener el objeto Producto en específico y usando el productController mostrarlo en pantalla con el GetMapping(“/{id}”) donde se va a cargar el modelo de update con los datos del producto con el id en cuestión. En el formulario creado en (producto/{id}) se utilizó Thymeleaf para manejar el método post, crear el objeto y cargar los campos usando los inputs del formulario. En el controller se crea el update que va a llamar al .add del productoService, este va a estar relacionado al método post del botón de Edit. Al enviar el formulario se activa el método postMapping(/{id}) para modificar objeto específico de tipo Producto en la base de datos Controller:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/controller/ProductoController.java	

	Service:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/service/ProductoService.java Vista:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/templates/producto/new.html

Curso normal

Nro	Ejecutor	Paso o actividad
1	Admin	Selecciona el botón Producto
2	Admin	Encuentra el producto a modificar
3	Admin	Selecciona el botón Modificar
4	Admin	Selecciona el/los campos a modificar
5	Admin	Selecciona el botón Edit
6	Sistema	Se actualiza el producto con los datos ingresados

Curso alternativo

Nro	Descripción de acciones alternas
7	El producto no se logra actualizar, redireccionando a un error. Referencia: Paso 6
8	Comunicarse con soporte técnico
9	Se ingresa un dato no correspondiente a alguno de los campos Referencia: Paso 6
10	El sistema pide que ingreses el tipo de dato correspondiente
11	El administrador selecciona el botón Edit
12	El sistema actualiza el producto con los datos ingresados

6 - Realizar baja de producto (Admin)

Commit	Ezequiel Gonzalez (Esqueleto, vista y fixes)	
Revisado/Modificado	Arturo Eduartes (capa Service) Tomás De Maio (capa Controller)	
Caso de uso	Dar Producto de Baja	Identificador: CU 6
Actores	Administrador	
Tipo	Primario	
Referencias	Iniciar sesión como administrador e ingresar a la pestaña Productos. Entonces en la pestaña productos apretar el botón de Modificar Producto y dentro del formulario cambiar el Option de valido de “True” a “False”. Caso de uso relacionado: CU1	
Precondición	Deben existir productos en el sistema	
Postcondición	Se genera una baja lógica sobre el producto	
Descripción	Se quiere eliminar un producto de los productos disponibles	
Resumen	Se creó un método findById(int id) en serviceInterfaces, el cual tiene el método findById por defecto en el productRepository para obtener el objeto Producto en específico y usando el productController mostrarlo en pantalla con el GetMapping(“/{id}”) donde se va a cargar el modelo de update con los datos del producto con el id en cuestión. En el formulario creado en (producto/{id}) se utilizó Thymeleaf para manejar el método post, crear el objeto y cargar los campos usando los inputs del formulario. En el controller se crea el update que va a llamar al .add del productoService, este va a estar relacionado al método post del botón de Edit. Al enviar el formulario se activa el método PostMapping(/{id}) subir el cambio del booleano de activo de 1 a 0 al producto en la Base de datos.	

	Controller:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/controller/ProductoController.java Service:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/service/ProductoService.java Vista:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/templates/producto/new.html

Curso normal

Nro	Ejecutor	Paso o actividad
1	Admin	Selecciona el botón Producto
2	Admin	Encuentra el producto a eliminar
3	Admin	Selecciona el botón Modificar
4	Admin	Selecciona el campo Activo
5	Admin	Selecciona la opción False
6	Admin	Selecciona el botón Edit
7	Sistema	Se elimina al producto de la lista de productos disponibles

Curso alternativo

Nro	Descripción de acciones alternas
8	El producto no se elimina de la lista de productos disponibles Referencia: Paso 7
9	Comunicarse con servicio técnico

7 - Realizar compra de producto (Cliente)

Commit	Tomás De Maio (Esqueleto y Controller)	
Revisado/Modificado	Ezequiel Gonzalez (Vista y fixes) Arturo Eduartes (Service y updates) Gaston Romero (updates)	
Caso de uso	Comprar Producto	Identificador: CU 7
Actores	Cliente	
Tipo	Primario	
Referencias	Caso de uso relacionado: CU1	
Precondición	Se debe haber iniciado sesión, acceder a la tabla de productos y deben existir productos disponibles para su compra	
Postcondición	El sistema registra la compra realizada y (Sistema) se hace la baja de la cantidad comprada del stock	
Descripción	El cliente desea comprar un producto	
Resumen	Se creó un método findProductosDisponibles() en productoRepository con el Query indicando que se obtengan en una lista todos los productos que tengan un stock restante mayor a 0 y que estén disponibles a su venta. Usando el productController mostrar todos los objetos en pantalla con el GetMapping(“/index”) donde se va a cargar toda la vista con los productos disponibles. Se ignoran las tablas de estadísticas ya que tienen la propiedad de (if: isAdmin). Una vez que se muestran en pantalla, el Cliente puede efectuar una compra del producto que desee, se puede seleccionar la cantidad deseada. Una vez finalizada la compra del producto, se baja el stock restante, y se registra la venta en su propio historial, para poder consultarlo posteriormente. Mientras se estaba en reunión la creación del caso de uso se hizo con la participación activa del grupo en cuanto a investigación y recomendaciones.	

	Controller:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/controller/ProductoController.java Service:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/service/ProductoService.java Vista:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/templates/producto/index.html

Curso normal

Nro	Ejecutor	Paso o actividad
1	Cliente	Selecciona el botón Productos
2	Cliente	Decide qué producto comprar
3	Cliente	Selecciona la casilla de cantidad
4	Cliente	Ingresa la cantidad a comprar
5	Cliente	Selecciona el botón Comprar
6	Sistema	Se reduce el stock restante en base a la cantidad

Curso alternativo

Nro	Descripción de acciones alternas
7	El cliente no encuentra el producto deseado. Referencia: Paso 2
8	El cliente selecciona el botón de logout

8 - Revisar compra de producto (Cliente)

Commit	Arturo Eduartes (Service y fixes)	
Revisado/Modificado	Ezequiel Gonzalez (Vista) Tomas Diaz De Maio (Controller y fixes) Gaston Romero (updates)	
Caso de uso	Comprar Producto	Identificador: CU 8
Actores	Cliente	
Tipo	Primario	
Referencias	Caso de uso relacionado: CU1 y CU2	
Precondición	El cliente debe haber realizado una compra previamente	
Postcondición	Ninguna	
Descripción	El cliente desea revisar la compra realizada	
Resumen	Se revisó en conjunto para ver cómo se podía implementar el sistema de compra de un producto por parte del cliente. Se ha implementado la funcionalidad de que un Cliente pudiera comprar un producto de la lista de opciones de la misma, además se puede seleccionar la cantidad que desea comprar, se efectúan las validaciones internamente en el sistema y al finalizar si todo está correcto (hay stock disponible y los datos de entrada son válidos) se realiza la venta y se registra en su historial de venta, la cual puede consultar cuando el Cliente lo desee. Se implementa fundamentalmente la corroboración de si es necesario ejecutar un Pedido Compra (CU 9). Mientras se estaba en reunión la creación del caso de uso se hizo con la participación activa del grupo en cuanto a investigación y recomendaciones. Controller:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/	

	oo2/grupo2/controller/ProductoController.java Service:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/service/ProductoService.java Vista:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/templates/producto/index.html

Curso normal

Nro	Ejecutor	Paso o actividad
1	Cliente	Selecciona el botón Historial
2	Cliente	Revisa si el producto es el correcto
3	Cliente	Revisa si la fecha es correcta
4	Cliente	Revisa si la cantidad es correcta
5	Cliente	Revisa si el importe es correcto

Curso alternativo

Nro	Descripción de acciones alternas
6	El cliente detecta que algún valor es incorrecto. Referencia: Paso 2,3,4,5.
7	El cliente se comunica con el administrador

9 - Realizar reposición de stock (Admin)

Commit	Arturo Eduartes (Esqueleto y Service)	
Revisado/Modificado	Tomás De Maio (capa Controller) Ezequiel Gonzalez (capa Vista) Gaston Romero (fixes)	
Caso de uso	Reponer Stock	Identificador: CU 9
Actores	Administrador	
Tipo	Primario	
Referencias	Iniciar sesión como administrador e ingresar a la pestaña Pedido Compras. Caso de uso relacionado: CU1	
Precondición	Deben existir productos en el sistema y estos deben tener una cantidad menor a 5	
Postcondición	Se genera el Pedido compra en el sistema y se registra en la tabla de compras de la pestaña Compras	
Descripción	Se quiere reponer stock de algún producto	
Resumen	Se revisó en conjunto cómo se podía implementar el sistema de la reposición de Stock de los productos. De manera tal que los productos de manera generalizada tienen una comprobación de 5 unidades, cuando se produce una compra de un Cliente, se comprueba si es inferior a 5 posterior a la compra, en caso de ser así se crea un Pedido Compra y se registra en la base de datos de manera interna sin notificar al Cliente ni tampoco al Admin, impidiendo un flujo poco fluido y dinámico. Cuándo ingresa el Admin, tiene la posibilidad de realizar la reposición de Stock de un producto. Por el cual se produjo un Pedido Compra, una vez que se realiza el abastecimiento, se agrega automáticamente la suma de cantidad de Stock en el producto solicitado el Pedido Compra de manera	

	interna por medio del sistema, mediante procedimientos. Mientras se estaba en reunión la creación del caso de uso con la participación activa del grupo en cuanto a investigación y recomendaciones. Controller: PedidoCompra:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/controller/PedidoCompraController.java Compra:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/controller/CompraController.java Service: PedidoCompra:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/service/PedidoCompraService.java Compra:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/service/CompraService.java Vista: PedidoCompra:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/templates/pedidocompra/index.html Compra:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/templates/venta/index.html

Curso normal

Nro	Ejecutor	Paso o actividad
1	Admin	Selecciona el botón Pedido Compras
2	Admin	Encuentra el producto a reponer en la tabla
3	Admin	Selecciona el botón Realizar Compra
4	Admin	Utiliza el slider para determinar la cantidad
5	Admin	Selecciona la casilla de proveedores

6	Admin	Elige el proveedor correcto
7	Admin	Selecciona el botón enviar
8	Sistema	Se repone stock en producto elegido

Curso alternativo

Nro	Descripción de acciones alternas
9	Se produce un error a la hora de reponer stock Referencia: Paso 8
10	Comunicarse con soporte técnico

10 - Realizar historial de ventas (Cliente/Admin)

Commit	Tomas De Maio (Esqueleto y Controller)	
Revisado/Modificado	Ezequiel Gonzalez (Vista y fixes) Arturo Eduartes (Service y fixes) Gaston Romero (updates)	
Caso de uso	Historial Ventas	Identificador: CU 9
Actores	Admin/Cliente	
Tipo	Primario	
Referencias	Iniciar sesión y seleccionar el botón Ventas(en caso de ser admin)/Historial(en caso de ser cliente)	
Precondición	Deben existir productos vendidos por el sistema/comprados por el cliente	
Postcondición	Ninguna	
Descripción	Se quiere revisar el historial de ventas	
Resumen	Se revisó en conjunto cómo se podía implementar el sistema de visualización de las ventas registradas. En el caso del Admin se verán todas las ventas de todos los Clientes. En el caso del Cliente se verán solamente las ventas registradas por el Cliente ingresado, de manera tal que pueda visualizar todas sus ventas registradas en la app. Mientras se estaba en reunión la creación del caso de uso se hizo con la participación activa del grupo en cuanto a investigación y recomendaciones. Controller:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/controller/VentaController.java Service:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/java/com/unla/oo2/grupo2/service/VentaService.java	

	Vista:https://github.com/EzequielDeiana/grupo-2-OO2-2024/blob/main/grupo-2/src/main/resources/templates/venta/index.html

Curso normal

Nro	Ejecutor	Paso o actividad
1	Admin /Cliente	Selecciona el botón Ventas/Historial
2	Admin /Cliente	Revisa si el producto vendido/comprado es el correcto
3	Admin /Cliente	Revisa si la fecha es correcta
4	Admin /Cliente	Revisa si la cantidad es correcta
5	Admin /Cliente	Revisa si el importe es correcto

Curso alternativo

Nro	Descripción de acciones alternas
6	El cliente detecta que algún valor es incorrecto. Referencia: Paso 2,3,4,5
7	El cliente se comunica con el administrador
8	El administrador detecta que algún valor es incorrecto. Referencia: Paso 2,3,4,5
9	Comunicarse con soporte técnico